

CHAPTER 9 · SYSTEM DESIGN INTERVIEW

設計網路爬蟲

Design a Web Crawler

從需求釐清、容量估算、高層次架構到深入設計的完整思路

什麼是網路爬蟲？

爬蟲 (Web Crawler)

又稱 Robot 或 Spider，是搜尋引擎用來發現網路上「新增或更新內容」的自動化程式。

內容可以是：網頁、圖片、影片、PDF 檔等多種格式。

運作方式：

先收集一批初始網頁，再持續追蹤頁面中的連結，藉此擴散發掘更多新內容，如此反覆循環。

複雜度取決於規模

從幾小時可完成的小型專案，到需要專職團隊持續迭代的巨型系統，規模與需求決定了設計的複雜程度。

本章將依循「理解問題 → 高層次設計 → 深入探討 → 總結」四步驟展開。

爬蟲的四大應用場景

1

搜尋引擎索引

最常見用途：收集網頁建立本地索引，如 Googlebot 之於 Google 搜尋引擎。

2

網頁存檔

為未來保存資料，如美國國會圖書館、EU Web Archive 長期典藏網站內容。

3

資料探勘

從爆炸性成長的網路內容中挖掘知識，例如金融機構下載股東會議與年報。

4

網頁監控

偵測著作權與商標侵權，如 Digimarc 利用爬蟲找出盜版內容。

需求釐清與設計範圍

與面試官釐清的關鍵問題

Q 用途？ A 搜尋引擎索引

Q 每月處理量？ A 10 億個網頁

Q 內容類型？ A 僅限 HTML

Q 是否含新增/修改頁面？ A 需要考慮

Q 是否需儲存 HTML？ A 需要，保存 5 年

Q 重複內容如何處理？ A 直接忽略

釐清需求同樣重要

即使題目看似簡單如「設計爬蟲」，你與面試官的假設也可能不同——先問清楚範圍，再開始設計。

下一步：釐清「好爬蟲」該具備哪些特性。

良好爬蟲的四大特性

可擴展性 Scalability

網路上有數十億網頁，須透過平行化讓爬取極度高效。

健壯性 Robustness

網路充滿陷阱：壞 HTML、無回應伺服器、當機、惡意連結，都要能妥善處理。

禮貌性 Politeness

短時間內不對同一網站發出過多請求，避免造成負擔。

可擴充性 Extensibility

系統靈活，未來要支援新內容類型（如圖片）時無需重新設計整個系統。

容量估算

10 億

每月下載網頁數

~400

平均 QPS（每秒頁數）

800

尖峰 QPS（2× 平均）

500 KB

平均網頁大小

500 TB

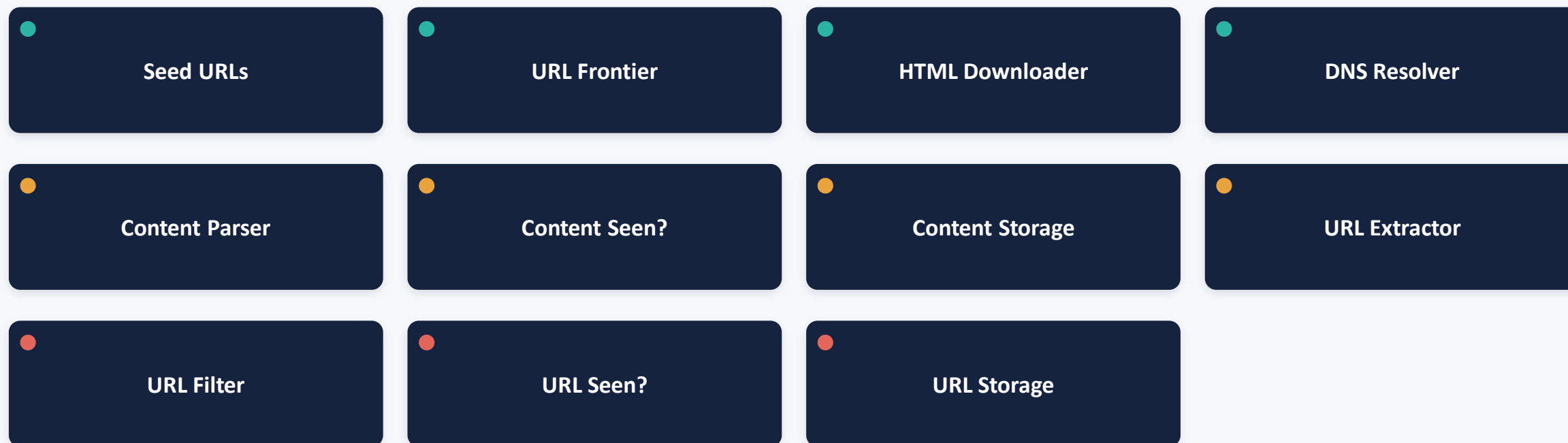
每月新增儲存量

30 PB

五年總儲存需求

計算邏輯：10 億 ÷ 30 天 ÷ 24 小時 ÷ 3600 秒 ≈ 400 頁/秒；500KB × 10 億 = 500TB/月；500TB × 12 個月 × 5 年 = 30PB。

高層次架構總覽



元件依「下載 → 解析 → 去重 → 抽取連結 → 過濾 → 回填佇列」流程串接，形成閉環。詳細資料流見下兩頁。

核心元件說明 — 資料進入端

1

Seed URLs

爬取起點。可依「地區」或「主題」（如購物、運動、醫療）劃分 URL 空間，讓爬蟲從好的起點盡量延伸。

2

URL Frontier

存放「待下載」URL 的佇列（FIFO），與「已下載」狀態分開管理，是控制優先權與禮貌性的核心元件。

3

HTML Downloader

依 URL Frontier 提供的清單，實際從網路下載網頁內容。

4

DNS Resolver

將 URL 轉換為 IP 位址，供 HTML Downloader 下載使用。

核心元件說明 — 解析與去重

5

Content Parser

驗證並解析下載的網頁，過濾異常/畸形頁面，獨立成元件避免拖慢下載流程。

6

Content Seen?

以「內容雜湊值」比對，避免重複內容多次入庫（研究顯示約 29% 網頁內容重複）。

7

Content Storage

儲存 HTML 內容：多數放在磁碟（容量大），熱門內容放記憶體（降低延遲）。

8

**URL Extractor / Filter / Seen? /
Storage**

抽取頁面連結 → 過濾黑名單與無效類型 → 判斷是否已處理過 → 新 URL 存回 Frontier。

爬蟲工作流程（11 步驟濃縮）

1

種子 URL 加入 Frontier

2

Downloader 取得待下載 URL

3

查詢 DNS 並開始下載

4

Content Parser 解析並驗證

5

送交 Content Seen? 判斷

6

重複則丟棄／否則送 Link
Extractor

7

抽取頁面中的連結

8

送交 URL Filter 過濾

9

送交 URL Seen? 判斷

10

已處理則略過

11

新 URL 寫回 URL Frontier, 循環
繼續

深入探討：DFS vs. BFS

可將網路視為有向圖：網頁是節點、超連結是邊。爬取過程即是圖的遍歷，常見演算法為 DFS 與 BFS。

DFS（深度優先）

通常不是好選擇 —— 遍歷深度可能非常深，容易失控。

BFS（廣度優先）

爬蟲普遍採用，透過 FIFO 佇列實作。但存在兩個問題：

- 同網站連結過多 → 集中對同一主機（如 wikipedia.com）大量請求，形同「不禮貌」的洗版行為。
- 未考慮 URL 優先權 → 網頁品質與重要性不同，理應依 PageRank、流量等排序。

深入探討：URL Frontier 三大機制

禮貌性 Politeness

同一主機一次僅下載一頁，並在兩次下載間加入延遲。透過「主機 → 佇列」對應表，讓每條佇列只含同一主機的 URL，再由 Queue Selector 分派給各 Worker Thread。

優先權 Priority

由 Prioritizer 依 PageRank、流量、更新頻率計算優先度；各優先權對應不同佇列，Queue Selector 以偏向高優先權的機率隨機選取。

新鮮度 Freshness

網頁不斷新增/刪除/編輯，需定期重新爬取。依「更新歷史」排程，並優先重爬重要頁面，而非全面重爬。

儲存策略：Frontier 中 URL 數量可達數億，採「磁碟為主、記憶體緩衝」的混合儲存，兼顧持久性與效能。

深入探討：HTML Downloader

Robots.txt（爬蟲排除協議）

網站用來告知爬蟲「哪些頁面可爬」的標準。下載前應先讀取並遵守規則，且將結果快取，避免重複下載同一份 robots.txt

。

範例（Amazon robots.txt 節錄概念）：對 Googlebot 禁止存取特定目錄，如評論與創作者後台路徑。

效能優化四招

分散式爬取 任務分散到多台伺服器、多執行緒，各自負責一部分 URL 空間。

DNS 快取 DNS 查詢同步阻塞、耗時 10–200ms，快取「網域 → IP」對應可大幅降低延遲。

地理位置就近部署 爬取伺服器靠近目標網站主機，下載速度更快。

設定逾時時間 伺服器回應慢或不回應時，設最長等待時間，逾時即放棄改爬其他頁面。

健壯性、擴充性與問題內容防範

健壯性 Robustness

- 一致性雜湊：協助分散負載，方便新增/移除下載伺服器
- 保存爬取狀態與資料，中斷後可載入回復
- 妥善的例外處理，避免整體系統崩潰
- 資料驗證，預防系統性錯誤

擴充性 Extensibility

以「模組插拔」方式支援新內容類型，例如新增 PNG Downloader 模組處理圖片，或加入 Web Monitor 模組防範侵權，皆不需重新設計整體系統。

偵測問題內容

- 重複內容：以雜湊/校驗和快速偵測（約 30% 頁面重複）
- 蜘蛛陷阱：無限深的目錄結構會使爬蟲陷入無窮迴圈，可限制 URL 最大長度並人工排查
- 資料雜訊：廣告、程式碼片段、垃圾連結等低價值內容應盡量排除

總結與延伸主題

本章回顧

我們從好爬蟲的四大特性出發（可擴展性、健壯性、禮貌性、可擴充性），提出高層次架構，並拆解每個元件與 11 步驟工作流程，最後深入 DFS/BFS、URL Frontier、Downloader 優化與健壯性設計。

尚未涵蓋的延伸主題

- 伺服器端渲染：因應 JavaScript／AJAX 動態產生的連結
- 過濾垃圾頁面：以反垃圾元件排除低品質內容
- 資料庫複寫與分片：提升資料層可用性與擴展性
- 水平擴展：伺服器須保持無狀態（stateless）
- 可用性、一致性與可靠性：大型系統成功的核心
- 數據分析：持續優化系統的重要依據

Q & A

謝謝聆聽 — 歡迎討論設計取舍與延伸應用